

Lista de Exercícios – Manipulação de Listas (append(), remove(), clear(), count())

Enunciado: Resolva os exercícios a seguir utilizando a linguagem Python, com foco na **manipulação de listas (vetores)** e no uso de métodos como `append()`, `remove()`, `clear()` e `count()`. Utilize a estrutura de repetição **for** (e, quando necessário, **while**) para percorrer os elementos das listas e realizar operações como **contagem, soma, busca e verificação de condições**. Sempre que necessário, utilize **contadores, acumuladores**, estruturas condicionais (**if**) e operadores relacionais e lógicos. Também utilize funções como `len()` e `range()` quando apropriado. Os dados podem ser informados pelo usuário por meio da função `input()` e armazenados em listas. Cada programa deve seguir a lógica de **entrada, processamento e saída**, apresentando os resultados de forma clara. Mantenha o código **organizado**, com boa indentação, nomes de variáveis claros e mensagens compreensíveis, facilitando a leitura e o entendimento do algoritmo.

1) Uma escola deseja organizar os dias úteis da semana em um sistema simples feito em Python. Para isso, será necessário criar inicialmente uma lista vazia e adicionar os dias manualmente conforme o cadastro for sendo realizado. Crie um programa que utilize o método `append()` para inserir, um por um, os seguintes valores na lista: 'segunda', 'terça', 'quarta', 'quinta' e 'sexta'. Após concluir todas as inserções, o programa deve exibir a lista completa para confirmar que todos os elementos foram armazenados corretamente e na ordem desejada.

2) Um professor está analisando a frequência com que determinados números aparecem em uma sequência de valores coletados durante uma atividade em sala. Considere a lista `numeros = [4, 7, 2, 7, 9, 3, 7, 1]` e desenvolva um programa que utilize o método `count()` para verificar quantas vezes o número 7 aparece dentro da lista. O resultado da contagem deve ser exibido de forma clara para o usuário, indicando exatamente quantas ocorrências foram encontradas.

3) Uma turma possui uma lista de alunos cadastrados em um sistema simples de gerenciamento escolar. Em determinado momento, pode ser necessário remover o nome de um aluno da lista, caso ele tenha sido cadastrado incorretamente ou transferido de turma. Crie um programa que possua inicialmente uma lista contendo 5 nomes de alunos. Em seguida, peça ao usuário que informe qual nome deseja remover utilizando o método `remove()`. Caso o nome exista na lista, ele deverá ser removido e a nova lista atualizada deverá ser exibida. Se o nome informado não existir, o programa deverá informar essa situação ao usuário.

4) Durante uma aula de programação, o professor pediu aos alunos que criassem uma lista contendo os números de 1 até 10 de forma automática, utilizando repetição. Faça um programa que comece com uma lista vazia e utilize um laço **for** juntamente com o método `append()` para adicionar os números de 1 a 10. Depois de exibir todos os valores armazenados na lista, utilize o método `clear()` para remover todos os elementos. Ao final, mostre a lista antes e depois da limpeza para comprovar que ela foi esvaziada corretamente.

5) Uma empresa está analisando a frequência de determinadas letras em uma sequência de caracteres armazenada em uma lista. Considere a lista `letras = ['a', 'b', 'a', 'c', 'a', 'd', 'b', 'a']` e desenvolva um programa que utilize o método `count()` para descobrir quantas vezes as letras 'a' e 'b' aparecem. Depois de realizar as contagens, o programa deverá exibir os resultados encontrados e informar qual das duas letras possui a maior quantidade de ocorrências. Caso ambas apareçam a mesma quantidade de vezes, essa informação também deverá ser exibida.

6) Uma secretaria escolar deseja criar um pequeno sistema para registrar os nomes dos alunos de uma turma. O programa deverá permitir que o usuário digite vários nomes, armazenando cada um deles em uma lista através do método `append()`. O cadastro deve continuar acontecendo até que o usuário digite a palavra 'fim', indicando que não deseja mais adicionar alunos. Após encerrar o cadastro, o programa deverá exibir a lista completa de alunos cadastrados e solicitar ao usuário um nome para ser removido utilizando o método `remove()`. Caso o nome exista, ele deverá ser excluído e a lista atualizada deverá ser mostrada na tela.

7) Um mercado está simulando o funcionamento de um carrinho de compras em um sistema simples. Considere a lista inicial `carrinho = ['leite', 'pão', 'queijo', 'manteiga']`. O sistema deverá remover o item 'pão' utilizando o método `remove()` e, logo em seguida, adicionar o item 'café' utilizando `append()`. Após cada modificação realizada no carrinho, a lista atualizada deverá ser exibida para que o usuário acompanhe as alterações feitas nos produtos armazenados.

8) Durante uma pesquisa, vários números inteiros serão digitados pelo usuário e armazenados em uma lista para análise posterior. Faça um programa que leia 8 números inteiros e armazene todos eles em uma lista utilizando `append()`. Depois de finalizar o preenchimento, o programa deverá identificar qual foi o primeiro número digitado e utilizar o método `count()` para descobrir quantas vezes esse mesmo número aparece ao longo da lista. Ao final, mostre o resultado da contagem ao usuário.

9) Uma pessoa deseja criar uma lista de compras digital simples para adicionar produtos conforme lembrar deles. Desenvolva um programa em que o usuário possa digitar itens que serão armazenados em uma lista utilizando o método `append()`. O processo de cadastro deve continuar até que a palavra 'sair' seja digitada. Após isso, o programa deverá exibir todos os itens cadastrados na lista, utilizar o método `clear()` para apagar completamente os dados armazenados e, por fim, utilizar `len()` para confirmar que a lista realmente ficou vazia.

10) Um sistema de organização de dados precisa analisar a frequência com que determinadas cores aparecem em uma lista. Considere a lista `cores = ['azul', 'verde', 'azul', 'vermelho', 'azul', 'verde']` e desenvolva um programa que utilize o método `count()` para verificar quantas vezes cada cor aparece. O programa deverá percorrer a lista, realizar as contagens necessárias e exibir separadamente a quantidade de ocorrências de cada cor encontrada.

11) Uma empresa deseja criar um pequeno sistema de gerenciamento de contatos utilizando listas em Python. O programa deverá apresentar um menu interativo permitindo ao usuário escolher diferentes operações:

- 1 - Adicionar contato (`append`)
- 2 - Remover contato (`remove`)
- 3 - Quantas vezes um nome aparece (`count`)
- 4 - Limpar todos os contatos (`clear`)
- 5 - Exibir lista de contatos
- 0 – Sair

O programa deve continuar em loop até o usuário escolher a opção 0. Use Case para fazer o menu de opções